

Praneesh R V

CB.SC.U4CYS23036

20CYS443 - Biometrics

Assignment 2 - Mapping the body technology

Q1) Body mapping technology and deepfake liveness detection

Body mapping in this context relies on pose estimation models such as OpenPose, MediaPipe Pose, or BlazePose, which detect a fixed set of skeletal keypoints (shoulders, elbows, wrists, hips, knees, ankles, and often facial landmarks) frame by frame from RGB video. These models use convolutional neural networks trained to output heatmaps for each joint location, then apply a part-affinity-field or similar association step to connect keypoints into a coherent skeleton per person. Once keypoints are tracked across the video sequence, the system derives a gait signature — features like stride length, cadence, joint-angle trajectories, hip sway, and vertical bounce — which together form a behavioral biometric distinct from any single static frame.

For liveness detection against a deepfake presented on a second screen, SecureID's pipeline typically runs several checks: (a) texture/frequency analysis to detect screen artifacts like moiré patterns, unnatural reflections, or GAN-typical frequency signatures; (b) challenge-response actions (blink on cue, turn head, read a random number) checked for correct timing and natural micro-movements; (c) depth or 3D-structure cues, since a flat screen lacks true depth parallax when the phone or head moves slightly; and (d) audio-visual sync checks for the spoken number. A high-resolution deepfake played on a second screen is most likely to defeat the challenge-response layer (a good deepfake can be pre-generated to blink and turn on cue, and audio can be synced), because that layer only validates *behavioral compliance*, not the physical presence of a live 3D subject. The pipeline is most likely to fail at the presentation attack detection (PAD) / depth-liveness stage if it lacks active depth sensing (structured light, ToF, or stereo parallax) — a 2D RGB-only liveness check is the weak point, since screen-based replay defeats texture and motion heuristics far more easily than it defeats a true depth check.

Q2) Data flow diagram and trust boundary for the malware camera-interception attack

Describe the data flow as: Camera Sensor → OS Camera Driver/API → App's Face-Detection Module → On-device Embedding Generation → Encrypted Transmission → Cloud Match Server → Template Database → Accept/Reject Decision.

The attack occurs where malware sits between "Camera Sensor/OS Camera API" and "App's Face-Detection Module" — this is the trust boundary between the operating system/hardware layer (assumed trusted by the app) and the application's input-processing layer. The app implicitly trusts that whatever the OS camera API hands it is a genuine, real-time feed; malware exploiting this boundary performs a frame-injection or camera-feed-hijack attack, substituting a pre-recorded video before the face-detection module ever sees the real camera output.

Control to prevent it: Implement hardware-backed attestation of the camera pipeline — e.g., using a Trusted Execution Environment (TEE) or Secure Camera API (such as Android's SafetyNet/Play Integrity combined with camera2 API's secure capture mode) that cryptographically signs frames at the sensor/driver level and verifies signature integrity before the face-detection module accepts them. This closes the gap by making it detectable if the feed was tampered with between capture and processing, rather than trusting the OS layer blindly.

Q3) What stolen embeddings enable, and a template protection scheme

Stolen biometric embeddings are vector representations (typically 128–512 dimensional floating-point vectors), not raw images, so an attacker cannot directly view the person's face or gait from the embedding alone, and cannot trivially reverse it to a photorealistic image without significant computational effort (see Q4).

However, an attacker can: (a) replay the embedding directly into the matching system if the system doesn't verify liveness at match time, achieving unauthorized authentication; (b) use the embedding for cross-matching against other breached databases to link identities across services (since raw embeddings from the same biometric are often similar across systems); (c) attempt model-inversion or reconstruction attacks to recover an approximate face; and (d) potentially deanonymize the person if the embedding is linked to other leaked metadata.

Template protection scheme — Cancelable Biometrics via Biohashing: Instead of storing the raw embedding, SecureID should transform it using a user-specific, revocable secret (a random projection matrix or salt seed stored separately, e.g., in a secure enclave or derived from a user PIN). The biohashing process multiplies the embedding vector by this secret transformation matrix and thresholds/quantizes the

result into a binary or transformed vector — this transformed template is what's stored. If breached, the transformed template is: (a) non-invertible back to the original embedding without the secret, (b) revocable — simply issue the user a new transformation seed and regenerate the template from a fresh enrollment, and (c) diversifiable — the same underlying biometric produces unlinkable templates across different services, preventing cross-database correlation. This directly limits breach damage because compromise of the stored template doesn't compromise the user's underlying biometric identity, and it can be "reset" unlike a raw fingerprint or face.

Q4) Credibility of model-inversion attacks and mitigation

This is a credible threat. Academic research (e.g., on face-recognition model inversion) has demonstrated that attackers with access to embeddings and knowledge of (or query access to) the same embedding model architecture can train a generative inversion network to reconstruct recognizable — not pixel-perfect but identifiable — face images from embeddings, especially when they have many embedding samples or white-box access to the encoder. Given SecureID stores embeddings centrally and a competitor claims capability, this is within reach of a moderately resourced adversary, particularly if the embedding model is a known/public architecture (e.g., a standard FaceNet-style model), which makes building an inversion model easier via transfer learning.

Architectural mitigation: Move to on-device matching with only match-scores or encrypted comparison results transmitted, rather than storing raw embeddings centrally at all (ties into the biohashing/cancelable transform from Q3, plus techniques like homomorphic encryption or secure multiparty computation for server-side matching without ever exposing the plaintext embedding server-side). Combining the cancelable-biometric transform (Q3) with never persisting the raw, invertible embedding removes the actual attack surface for inversion, since there is no raw embedding to invert — the attacker would need to invert the *transformed* template, which is designed to be one-way.

Q5) STRIDE analysis of gait mimicry attack

The attacker studies the legitimate user's gait from stolen social-media videos and physically mimics it while holding the victim's phone during the KYC gait-capture step. This falls under Spoofing in the STRIDE model. Justification: Spoofing is defined as illegitimately assuming the identity of another entity/user. Here the attacker is not tampering with data in transit, not repudiating an action, not causing information disclosure through the attack itself, not denying service, and not elevating privilege through a technical exploit — they are purely presenting a *false*

identity claim to the biometric authentication system by imitating the physical behavioral trait (gait) that the system uses as an identity proxy. This is the classic definition of a presentation/spoofing attack in biometric systems — analogous to a photo spoof against face recognition, just applied to a behavioral rather than physiological modality. A secondary, weaker argument could touch Tampering if the attacker also manipulates sensor input, but the core threat as described is Spoofing.

Q6) Before/after attack tree: cloud vs. on-device matching

Root goal: "Impersonate legitimate user during KYC"

Before (cloud-based matching) — attack paths:

1. Intercept camera feed on-device (malware) → inject pre-recorded video before face-detection.
2. Intercept embedding in transit between device and cloud (MITM if transport security is weak) → replay a stolen embedding directly to the matcher.
3. Compromise the cloud server/database → exfiltrate templates, then replay valid embeddings directly at the API layer, bypassing capture entirely.
4. Compromise the transmission channel to inject a synthetic embedding without ever touching a real camera.

After (on-device matching) — attack paths:

1. Intercept camera feed on-device (malware) → inject pre-recorded video before face-detection (same as before — this path is *not* eliminated by moving matching on-device).
2. Reverse-engineer/tamper with the on-device matching binary to force an "accept" decision locally (new attack surface introduced by edge processing — attacker now targets the device's app integrity/root-of-trust instead of the network).
3. Extract the on-device model/templates via device rooting/jailbreak to attempt offline replay or inversion attacks.

Argument: On-device matching removes the network-interception and central-database-breach paths (paths 2, 3, 4 in the "before" tree), which are high-impact because a single cloud breach compromises *all* users at once. However, it concentrates risk onto the device itself and introduces new local-tampering paths. On balance, on-device matching has lower residual risk provided it's paired with device attestation (TEE/secure enclave for the matching logic) and root/jailbreak detection — because a compromised database is a catastrophic, scalable breach (all

users at once), while a compromised individual device is an isolated, non-scalable breach limited to that user. The residual risk is lower in aggregate even though it is not zero, so the recommendation should pair edge processing with hardware-backed integrity attestation.

Q7) Regulatory risk points: data retention, consent, breach response

As the security analyst, the key points to raise:

Retention: Biometric templates should be retained only as long as strictly necessary for the KYC purpose and any applicable regulatory record-keeping period (e.g., AML/KYC retention mandates), not indefinitely. A retention schedule with automatic deletion/expiry must be defined and enforced, and raw video/audio (as opposed to derived embeddings) should be deleted immediately after embedding extraction rather than retained at all, since raw biometric data carries far higher sensitivity and reconstruction risk.

Consent: Consent must be specific, informed, and separable from general terms-of-service — the user must be clearly told that facial video and gait/motion data will be captured, what it will be used for (identity verification only vs. any secondary use like fraud analytics), and must have a genuine ability to withdraw consent and request deletion. Because biometric data is typically classified as a "special/sensitive category" under most data protection regimes, explicit opt-in consent (not implied or bundled consent) is generally required, and the consent flow should be auditable.

Breach response: Because biometric templates cannot be reset like a password, the breach-response plan needs to address irrevocability directly — this is the central regulatory question. SecureID should raise: (a) what compensating control exists if templates leak (this is exactly why the cancelable-biometrics/biohashing scheme from Q3 matters — it makes the stored artifact revocable even though the underlying biometric is not); (b) mandatory breach notification timelines to regulators and affected users; (c) a clear incident-response runbook specifically for biometric-data compromise (distinct from a generic password breach) including re-enrollment procedures; and (d) whether affected users can be meaningfully protected post-breach at all, or whether the harm is effectively permanent — which itself may be a point regulators specifically probe.

Q8) Spike in failed liveness attempts from a single IP range

This pattern — a sudden burst of API calls to the video-upload endpoint from one IP range, none passing the liveness challenge — most likely represents an automated

credential-stuffing / brute-force presentation-attack campaign, where an attacker (or botnet) is systematically submitting spoofed video/photo/deepfake attempts at scale, probably testing which spoofing technique or which stolen identity data can slip past the liveness check, or simply probing the endpoint for a vulnerability/timing weakness.

STRIDE mapping: This maps primarily to Denial of Service (the volume of malformed/failing requests can degrade endpoint availability for legitimate users and consume backend compute for video/liveness processing) with a secondary Spoofing element (each individual attempt is itself an identity-spoofing attempt). Given the question asks for "the correct" category and emphasizes volume/pattern over content, Denial of Service is the primary classification, since the defining signal here is abnormal request volume behavior, not a single successful or crafted spoof.

Technical control: Implement rate limiting and adaptive throttling at the API gateway, keyed by IP range/device fingerprint, with a progressive lockout (e.g., exponential backoff after N failed liveness attempts within a time window) plus integration with a Web Application Firewall (WAF) or bot-detection service to flag and block the offending IP range, alongside CAPTCHA or proof-of-work challenges inserted before allowing further liveness attempts from that source.

Q9) Multimodal fusion: feature-level vs. decision-level, and attack surface

Feature-level fusion combines the raw feature vectors/embeddings from face, gait, and voice into a single joint representation *before* the matching decision is made. This means an attacker attempting to spoof all three modalities must produce inputs that jointly satisfy the fused feature space simultaneously and consistently — a partial spoof (e.g., a good deepfake face but an inconsistent voice) is likely to push the combined feature vector far enough from the genuine template that the match fails outright, because errors compound multiplicatively across modalities within one decision. This raises the bar for the attacker significantly, but it also means the system is more brittle to legitimate variation (e.g., a cold affecting voice) and the fused model itself becomes a single, richer target — if the attacker can compromise the fusion model or its embedding space (e.g., via adversarial perturbation crafted against the joint representation), a single well-crafted attack could defeat all three modalities together.

Decision-level fusion runs each modality (face, gait, voice) through independent matchers, each producing its own accept/reject or confidence score, which are then combined via a rule (majority vote, weighted sum, AND/OR logic) to reach the final decision. This means an attacker can attack each modality independently and doesn't need a single unified spoof — they could, for example, focus effort on spoofing the two "easiest" modalities and rely on threshold/voting logic (if it's

OR-based or majority-based rather than requiring all three) to pass overall. This is generally a *larger* attack surface per-modality (each matcher can be probed and attacked separately, and each modality's spoofing techniques from prior questions remain individually valid), but it is more transparent/debuggable and lets SecureID apply modality-specific liveness defenses independently, and a strict AND-based decision fusion (all three must pass) can still be quite strong, at the cost of higher false-rejection rates for genuine users.

Conclusion: Feature-level fusion generally offers a *smaller* attack surface against simultaneous multi-modal spoofing (harder for an attacker to defeat all three at once through a single fused representation) but concentrates risk into the fusion model itself; decision-level fusion is more modular and easier to harden per-modality but is vulnerable to voting-logic weaknesses if not configured as a strict AND/high-threshold requirement across all three modalities.

Q10) Executive risk summary — one-page format

Structure this as a short paragraph plus a likelihood × impact narrative for five threats, since you want it in prose rather than table form:

Top 5 threats identified across Q1–9:

1. Deepfake/presentation-attack spoofing during onboarding (Q1, Q5) —
Likelihood: High (deepfake tooling is increasingly accessible). Impact: High (direct fraudulent account creation, financial and reputational loss).
2. On-device malware camera-feed interception (Q2) — Likelihood: Medium (requires device compromise, but malware-as-a-service makes this achievable at scale). Impact: High (bypasses liveness entirely, undetectable without hardware attestation).
3. Centralized biometric database breach and embedding exfiltration (Q3, Q4) —
Likelihood: Medium (insider threat plus external attackers targeting high-value biometric stores). Impact: Critical (irrevocable, affects all users simultaneously, severe regulatory exposure).
4. Automated/scaled presentation-attack probing against the API (Q8) —
Likelihood: High (already observed as an active pattern in the scenario).
Impact: Medium-High (service degradation plus eventual successful spoof if defenses aren't tuned).
5. Regulatory/consent and irrevocability exposure (Q7) — Likelihood: Medium (regulatory scrutiny is increasing globally for biometric KYC). Impact: High (fines, forced service suspension, loss of banking partner trust).

